

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Сложные методы поиска»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил студент группы РИС-25-26

Абрамова Валерия Андреевна

Принял: Доц. Полякова О.А.

Пермь 2026

Поиск методом Бойера-Мура (Бойера-Мура-Хорспула)

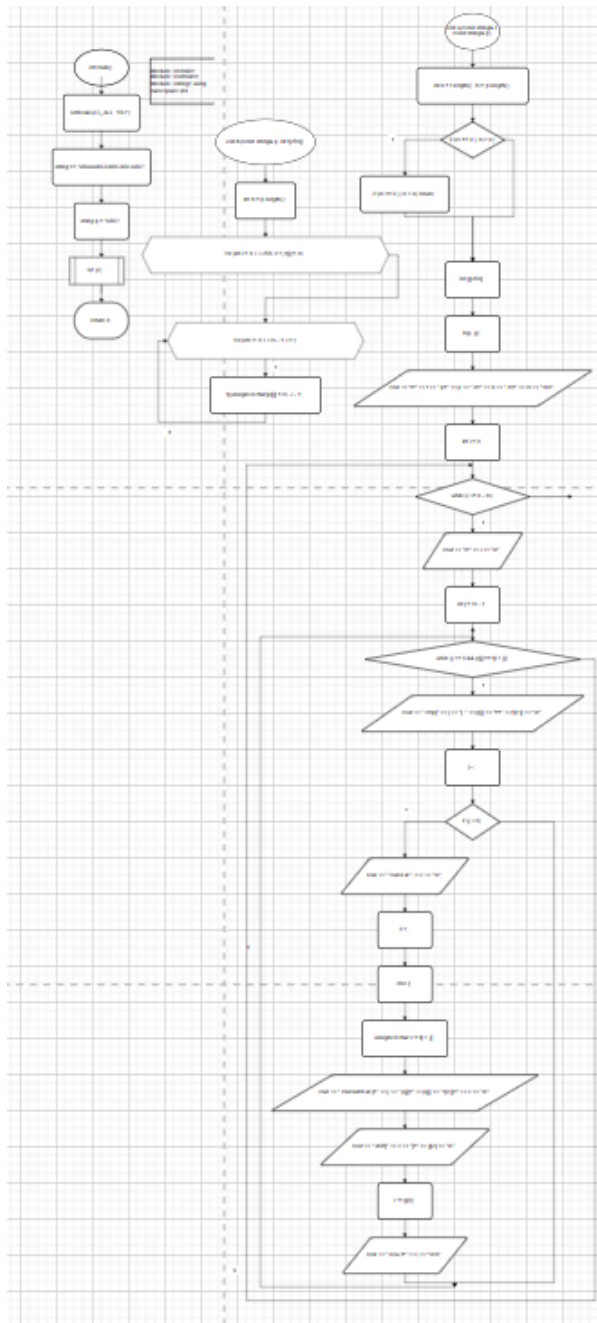
Постановка задачи

Определить индекс первого символа подстроки.

Анализ задачи

1. Создаём таблицу смещений на 256 символов. Заполняем её длиной образца m . Для каждого символа образца (кроме последнего) записываем смещение $= m - \text{позиция} - 1$.
2. Устанавливаем указатель $i = 0$. Пока $i \leq n - m$, сравниваем образец с текстом справа налево. Если все символы совпали — выводим позицию i и сдвигаем i на 1. Если несовпадение на символе s — берём из таблицы смещение для s и сдвигаем i на это значение. Повторяем, пока не дойдём до конца текста.

Блок-схема



Код

```
#include <locale>
#include <iostream>
#include <string>

using namespace std;

void b(const string& p, int t[256]) {
    int m = p.length();
    cout << "Образец: " << p << " (длина = " << m << ")\n\n";

    for (int i = 0; i < 256; i++) t[i] = m;

    cout << «Заполняем таблицу для символов образца (кроме последнего):\n»;
    for (int i = 0; i < m - 1; i++) {
        t[(unsigned char)p[i]] = m - i - 1;
        cout << « Символ '» << p[i] << «' на позиции « << i
            << « смещение = « << m - i - 1 << «\n»;
    }
    cout << « Остальные символы (в т.ч. последний '» << p[m - 1]
```

```

        << «') смещение = « << m << «\n\n»;
    }

void s(const string& t, const string& p) {
    int n = t.length(), m = p.length();
    cout << "строка: " << t << "\n";
    cout << "образец: " << p << "\n";
    cout << string(40, '=') << "\n";

    if (m == 0 || m > n) {
        cout << «образец пуст или длиннее текста\n»;
        return;
    }

    int g[256];
    b(p, g);

    int I = 0;
    int step = 1;

    while (I <= n - m) {
        cout << «\nШАГ « << step++ << «: Позиция в тексте i = « << i << «\n»;
        cout << " Текст: " << t << "\n";
        cout << " Сдвиг: " << string(I, ' ') << p << "\n";
        cout << « Сравнение справа налево:\n»;

        int j = m - 1;
        while (j >= 0 && p[j] == t[I + j]) {
            cout << " Символ " << j << ": " << p[j] << " == " << t[I + j] << "
\n";
            j--;
        }

        if (j < 0) {
            cout << « ПОЛНОЕ СОВПАДЕНИЕ на позиции « << i << « !\n»;
            cout << « Выводим позицию: « << i << «\n»;
            cout << " Сдвигаемся на 1 (i++): " << I << " → " << I + 1 << "\n";
            i++;
        }
        else {
            unsigned char c = t[I + j];
            cout << « НЕСОВПАДЕНИЕ: символ '» << p[j] << «' != '» << c << «'\n»;
            cout << « Стоп-символ: '» << c << «' на позиции « << i + j << « в
тексте\n»;
            cout << « Смещение из таблицы для '» << c << «': « << g[c] << «\n»;
            int shift = g[c];
            cout << " Было I = " << I << ", сдвиг = " << shift;
            I += shift;
            cout << " стало I = " << I << "\n";
        }

        if (I <= n - m) {
            cout << " Текущая позиция I = " << I << " (помещается)\n";
        }
        else {
            cout << " I = " << I << " > n-m = " << n - m << " выход\n";
        }
    }
}

int main() {
    setlocale(LC_ALL, "RU");
    string t = "АБАААВСДАВСАВСАВС";
    string p = "ABC";

    s(t, p);
    cout << endl;
}

```

```

    }
    return 0;
}

```

```

Заполняем таблицу для символов образца (кроме последнего):
Символ 'A' на позиции 0 смещение = 2
Символ 'B' на позиции 1 смещение = 1
Остальные символы (в т.ч. последний 'C') смещение = 3

ШАГ 1: Позиция в тексте i = 0
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    НЕСОВПАДЕНИЕ: символ 'C' != 'A'
Стоп-символ: 'A' на позиции 2 в тексте
Смещение из таблицы для 'A': 2
Было i = 0, сдвиг = 2 стало i = 2
Текущая позиция i = 2 (помещается)

ШАГ 2: Позиция в тексте i = 2
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    НЕСОВПАДЕНИЕ: символ 'C' != 'A'
Стоп-символ: 'A' на позиции 4 в тексте
Смещение из таблицы для 'A': 2
Было i = 2, сдвиг = 2 стало i = 4
Текущая позиция i = 4 (помещается)

ШАГ 3: Позиция в тексте i = 4
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    Символ 2: 'C' == 'C' ?
    Символ 1: 'B' == 'B' ?
    Символ 0: 'A' == 'A' ?
ПОЛНОЕ СОВПАДЕНИЕ на позиции 4 !
Выводим позицию: 4
Сдвигаемся на 1 (i++): 4 ? 5
Текущая позиция i = 5 (помещается)

ШАГ 4: Позиция в тексте i = 5
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    НЕСОВПАДЕНИЕ: символ 'C' != 'D'
Стоп-символ: 'D' на позиции 7 в тексте
Смещение из таблицы для 'D': 3
Было i = 5, сдвиг = 3 стало i = 8
Текущая позиция i = 8 (помещается)

ШАГ 5: Позиция в тексте i = 8
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    Символ 2: 'C' == 'C' ?
    Символ 1: 'B' == 'B' ?
    Символ 0: 'A' == 'A' ?
ПОЛНОЕ СОВПАДЕНИЕ на позиции 8 !
Выводим позицию: 8
Сдвигаемся на 1 (i++): 8 ? 9
Текущая позиция i = 9 (помещается)

ШАГ 6: Позиция в тексте i = 9
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    НЕСОВПАДЕНИЕ: символ 'C' != 'A'
Стоп-символ: 'A' на позиции 11 в тексте
Смещение из таблицы для 'A': 2
Было i = 9, сдвиг = 2 стало i = 11
Текущая позиция i = 11 (помещается)

ШАГ 7: Позиция в тексте i = 11
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    Символ 2: 'C' == 'C' ?
    Символ 1: 'B' == 'B' ?
    Символ 0: 'A' == 'A' ?
ПОЛНОЕ СОВПАДЕНИЕ на позиции 11 !
Выводим позицию: 11
Сдвигаемся на 1 (i++): 11 ? 12
Текущая позиция i = 12 (помещается)

ШАГ 8: Позиция в тексте i = 12
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    НЕСОВПАДЕНИЕ: символ 'C' != 'A'
Стоп-символ: 'A' на позиции 14 в тексте
Смещение из таблицы для 'A': 2
Было i = 12, сдвиг = 2 стало i = 14
Текущая позиция i = 14 (помещается)

ШАГ 9: Позиция в тексте i = 14
Текст:  АВАААВCDАВCАВCАВC
Сдвиг:   АВС
Сравнение справа налево:
    Символ 2: 'C' == 'C' ?
    Символ 1: 'B' == 'B' ?
    Символ 0: 'A' == 'A' ?
ПОЛНОЕ СОВПАДЕНИЕ на позиции 14 !
Выводим позицию: 14
Сдвигаемся на 1 (i++): 14 ? 15
i = 15 > n-m = 14  выход

```

Результат выполнения программы

Графическое решение

Шаг	i	Действие	Сдвиг
1	0	Несовпадение на А	2
2	2	Несовпадение на А	2
3	4	НАЙДЕНО на 4	1
4	5	Несовпадение на D	3
5	8	НАЙДЕНО на 8	1
6	9	Несовпадение на А	2
7	11	НАЙДЕНО на 11	1
8	12	Несовпадение на А	2
9	14	НАЙДЕНО на 14	1
10	15	Конец (i > n-m)	—

Алгоритм Кнута-Морриса-Пратта

Постановка задачі

Определить индекс первого символа подстроки.

Анализ задачи

Мы идём по тексту слева направо и одновременно сравниваем символы образца.

Когда символы совпадают → двигаемся дальше по образцу и по тексту.

Когда символ не совпадает $\rightarrow$ не возвращаемся назад по тексту, а используем префикс-функцию:

- Смотрим, сколько символов в начале образца уже совпало с концом текущего куска
- Сдвигаем образец так, чтобы эта совпавшая часть осталась на месте
- Продолжаем сравнение с того же места в тексте

Когда нашли полное совпадение → запоминаем позицию и продолжаем искать дальше.

Код

```
#include <locale>
#include <iostream>
#include <string>

using namespace std;

void KMP(const string& text, const string& pattern) {
    int n = text.length();
    int m = pattern.length();

    if (m == 0 || m > n) return;

    int pi[1000] = { 0 };
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && pattern[i] != pattern[j]) {
            j = pi[j - 1];
        }
        if (pattern[i] == pattern[j]) j++;
        pi[i] = j;
    }

    int j = 0;

    cout << "\nТекст:   " << text << endl;
    cout << "Образец:  " << pattern << "\n" << endl;

    for (int i = 0; i < n; i++) {
        while (j > 0 && text[i] != pattern[j]) {
            j = pi[j - 1];
        }
        if (text[i] == pattern[j]) j++;

        cout << "Позиция " << i << ": ";
        for (int k = 0; k <= i; k++) {
            cout << text[k];
        }
        cout << " | ";

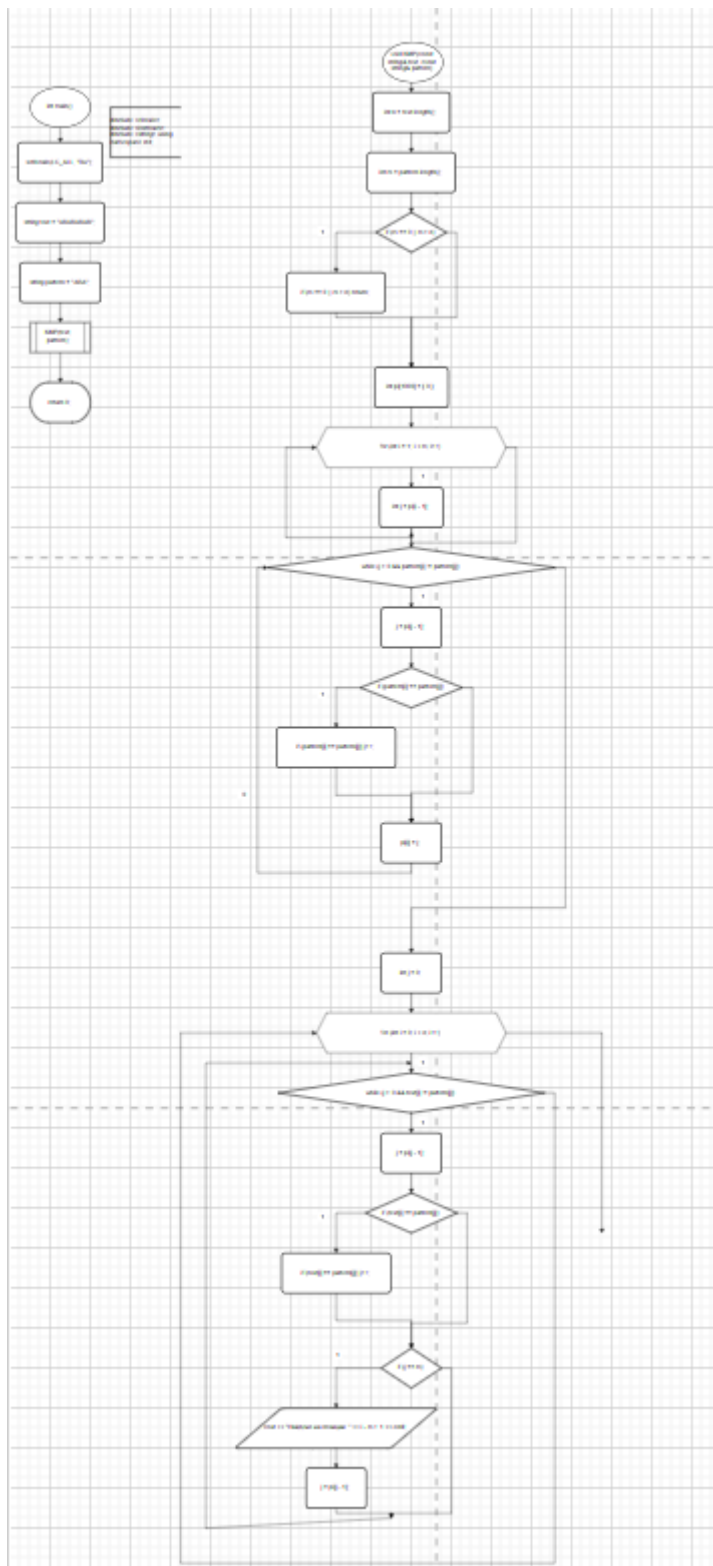
        for (int k = 0; k < j; k++) {
            cout << pattern[k];
        }
        cout << " (совпало " << j << "/" << m << ")";

        if (j == m) {
            cout << " <<< НАШЛИ! на позиции " << i - m + 1;
            j = pi[j - 1];
        }
        cout << endl;
    }
}

int main() {
```

```
    setlocale(LC_ALL, "Ru");  
  
    string text = "ABABABAB";  
    string pattern = "ABA";  
  
    KMP(text, pattern);  
  
    return 0;  
}
```

Блок-схема



Вывод программы

Текст: АВАВАВАВ

Образец: АВА

Позиция 0: А | А (совпало 1/3)

Позиция 1: АВ | АВ (совпало 2/3)

Позиция 2: АВА | АВА (совпало 3/3) <<< НАШЛИ! на позиции 0

Позиция 3: АВАВ | АВ (совпало 2/3)

Позиция 4: АВАВА | АВА (совпало 3/3) <<< НАШЛИ! на позиции 2

Позиция 5: АВАВАВ | АВ (совпало 2/3)

Позиция 6: АВАВАВА | АВА (совпало 3/3) <<< НАШЛИ! на позиции 4

Позиция 7: АВАВАВАВ | АВ (совпало 2/3)

Графическое решение